CSE-221: Algorithms

Practice Problem Set — Theory of Algorithmic Complexity

Department of Computer Science & Engineering, BRAC University
Md. Irtiza Hossain ext.irtiza.hossain@bracu.ac.bd

Topics: Asymptotic Notation (O, Ω, Θ, o, ω), Time Complexity of iterative code, Space Complexity. Problems drawn from CLRS Ch. 3.

Disclaimer: This is a practice problem set. There is no guarantee that these types of problems will appear in any exam. These problems are designed solely for your development of problem-solving skills.

Part A Asymptotic Notation

Problem 1

Using the formal definition of O-notation, show that $4n^2 + 100n + 500 = O(n^2)$.

Problem 2

Show that $20n^3 + 7n + 1000 = \Theta(n^3)$ by finding explicit constants c_1, c_2, and n_0.

Problem 3

Prove that $n^3 - 100n^2 \neq O(n^2)$.

Problem 4

Show that $1000n^2 + 1000n = O(n^2)$, that $n = O(n^2)$, but that $n^2 \neq O(n)$.

Problem 5

For each pair, state whether $f(n) = O(g(n))$, $\Omega(g(n))$, or $\Theta(g(n))$, and justify briefly.

(a) $f(n) = 3n^2 + 7n, \quad g(n) = n^2$
(b) $f(n) = n^3, \quad g(n) = n^2$
(c) $f(n) = \log_2 n, \quad g(n) = \log_{10} n$
(d) $f(n) = 2^{n+1}, \quad g(n) = 2^n$

Problem 6

For each pair below, determine the tightest correct asymptotic relation (O, Ω, or Θ).

(a) $f(n) = \dfrac{n^2}{2} - n, \quad g(n) = n^2$
(b) $f(n) = \log(n!), \quad g(n) = n \log n$
(c) $f(n) = (6n^3 + 1)\log n \;/\; (n + 1), \quad g(n) = n^2 \log n$
(d) $f(n) = n \log n + n, \quad g(n) = \log n$

Problem 7

Apply the simplified rules of O-notation to reduce each expression to its simplest asymptotic form. State the Θ bound where possible.

(a) $O(an^2 + bn + c)$, where a, b, c are positive constants.
(b) $O(n) + O(n^2)$
(c) $O(n) \cdot O(n)$
(d) $O(m) \cdot O(n) \cdot O(p)$
(e) $5 \cdot O(1)$
(f) $n \cdot O(1)$
(g) $O(n^2) + O(n \log n) + O(n)$

Problem 8

Let $f(n)$ and $g(n)$ be asymptotically positive. Prove or disprove each claim.

(a) If $f(n) = O(g(n))$, then $g(n) = \Omega(f(n))$.
(b) $f(n) + g(n) = \Theta(\max\{f(n), g(n)\})$.
(c) If $f(n) = O(g(n))$, then $2^{f(n)} = O(2^{g(n)})$.
(d) $f(n) + o(f(n)) = \Theta(f(n))$.

Problem 9

Explain why the statement "the running time of algorithm A is *at least* $O(n^2)$" is meaningless. What should it say instead?

Problem 10

Insertion sort has worst-case running time $\Theta(n^2)$ and best-case running time $\Theta(n)$.

(a) Is it correct to say its running time is $O(n^2)$? Justify.
(b) Is it correct to say its running time is $\Theta(n^2)$? Justify.
(c) Merge sort runs in $\Theta(n \log n)$ in all cases. Why can we drop the case qualifier here?

Problem 11

Rank the following functions by order of growth from slowest to fastest. Group functions that are Θ of each other.

$$1, \quad \log\log n, \quad \log n, \quad n^{1/3}, \quad \sqrt{n}, \quad n, \quad n\log n, \quad n^2, \quad n^3, \quad 2^n, \quad n!$$

Part B Time Complexity of Iterative Code

Problem 12

Find $T(n)$ and state the Θ bound.

```
int sum = 0;
for (int i = 1; i <= n; i++)
    sum = sum + i;            // O(1)
```

Problem 13

What is the overall time complexity? Analyse each fragment then combine.

```
// Fragment A
for (int i = 0; i < n; i++)
    a[i] = i * i;                        // O(1)

// Fragment B  (runs after A)
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        b[i][j] = a[i] + a[j];       // O(1)
```

Problem 14

Find $T(n)$ by counting inner iterations exactly, then give Θ.

```
int sum = 0;
for (int j = 1; j <= n; j++)
    for (int i = 1; i <= j; i++)
        sum++;
```

Problem 15

Determine $T(n)$. The outer loop multiplies by 2 each iteration; the inner always runs n times.

```
int sum = 0;
for (int k = 1; k <= n; k = k * 2)
    for (int j = 1; j <= n; j++)
        sum++;
```

Problem 16

The inner loop bound depends on `k`, not `n`. Find the exact sum and give Θ.

```
int sum = 0;
for (int k = 1; k <= n; k = k * 2)
    for (int j = 1; j <= k; j++)
        sum++;
```

Problem 17

Analyse the following loop with a `break`. Give the best-case, worst-case, and overall O bound.

```
for (int j = 0; j < n; j++) {
    // 3 O(1) operations
    if (someCondition(j)) break;
}
```

Problem 18

Find the time complexity of the following if-else. Identify best and worst case.

```
if (n % 2 == 0) {
    x = 0;                              // O(1)
} else {
    for (int i = 0; i < n; i++)
        x = x + i;                      // O(1) body
}
```

Problem 19

This is the SUM-ARRAY algorithm from CLRS. Find $T(n)$ by assigning costs to each line.

```
// SUM-ARRAY(A, n)
int sum = 0;                            // line 1
for (int i = 1; i <= n; i++)            // line 2
    sum = sum + A[i];                   // line 3
return sum;                             // line 4
```

Problem 20

Find $T(n)$ for the following triple-nested loop. Set up the exact sum and evaluate.

```
int count = 0;
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j++)
        for (int k = 1; k <= n; k++)
            count++;
```

Problem 21

Analyse this loop carefully — the bound on the inner loop is i, not n.

```
int sum2 = 0;
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i; j++)
        for (int k = 1; k <= j; k++)
            sum2++;
```

Problem 22

Find $T(n)$ for sequential search on an unsorted array. State both the worst-case and best-case separately, and give the O bound.

```
// Search array a[] of size n for value x
boolean seqSearch(int[] a, int n, int x) {
    for (int i = 0; i < n; i++)
        if (a[i] == x) return true;
    return false;
}
```

Part C Space Complexity

Problem 23

Find the **time** and **auxiliary space** complexity of the following.

```
int[] reverse(int[] a, int n) {
    int[] b = new int[n];              // allocates n integers
    for (int i = 0; i < n; i++)
        b[i] = a[n - 1 - i];
    return b;
}
```

Problem 24

Find the **time** and **auxiliary space** complexity of standard matrix multiplication. Only `i`, `j`, `k` are declared locally; `A`, `B`, `C` are given.

```
void matMul(int[][] A, int[][] B, int[][] C, int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            C[i][j] = 0;
            for (int k = 0; k < n; k++)
                C[i][j] += A[i][k] * B[k][j];
        }
}
```

Problem 25

Find the **time** and **auxiliary space** complexity of the following prefix-sum algorithm.

```
int[] prefixSum(int[] a, int n) {
    int[] p = new int[n];              // output array
    p[0] = a[0];
    for (int i = 1; i < n; i++)
        p[i] = p[i-1] + a[i];
    return p;
}
```

Problem 26

The following algorithm finds the maximum element in an array. Determine its **time** and **auxiliary space** complexity, including a careful account of every variable declared.

```
int findMax(int[] a, int n) {
    int max = a[0];                    // 1 variable
    for (int i = 1; i < n; i++)
        if (a[i] > max) max = a[i];
    return max;
}
```

Problem 27

The following 2-D array initialisation allocates a new $n \times n$ matrix. Find both the **time** and **total space** (not just auxiliary) complexity.

```
int[][] makeIdentity(int n) {
    int[][] I = new int[n][n];        // n x n matrix, all zeros
    for (int i = 0; i < n; i++)
        I[i][i] = 1;
    return I;
}
```

Problem 28

Binary search on a sorted array. Find the worst-case **time** complexity and the **auxiliary space** complexity (the array itself is input — count only locally declared variables).

```
int binarySearch(int[] a, int n, int x) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if      (a[mid] == x) return mid;
        else if (a[mid] <  x) low  = mid + 1;
        else                  high = mid - 1;
    }
    return -1;
}
```

Problem 29

The following algorithm counts inversions in an array using a 2-D scan. Find the **time** and **auxiliary space** complexity.

```
int countInversions(int[] a, int n) {
    int count = 0;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (a[i] > a[j]) count++;
    return count;
}
```

Problem 30

Bubble sort makes multiple passes and swaps adjacent elements. Determine its **time** complexity (best and worst case) and **auxiliary space**.

```
void bubbleSort(int[] a, int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)
            if (a[j] > a[j+1]) {
                int tmp = a[j];       // O(1) swap
```

Problem 30 (cont.)

```
6                   a[j]    = a[j+1];
7                   a[j+1]  = tmp;
8               }
9   }
```